A Full-Scale Autonomous Vehicle Platform for Engineering Education

Mohammadparsa Ghasemi, Behnam Bahr, and Viviane Seyranian
California State Polytechnic University, Pomona
Pomona, CA, USA

***Abstract*—This evidence-based practice paper presents an open-architecture autonomous vehicle platform designed to address the well-documented gap between academic preparation and industry requirements for cyber-physical systems engineers.**

The autonomous vehicle industry's rapid expansion has created unprecedented demand for engineers who can integrate sensing, computation, and actuation into safety-critical systems. Yet industry leaders consistently report that new graduates, while technically competent in isolated domains, lack the systems-level integration experience essential for autonomous systems development. A National Academies study documented this gap, noting that major employers require substantial internal training to bring new hires to full productivity in integrated systems roles [2]. Current educational approaches each carry significant limitations: simulation platforms abstract away real-world complexity including sensor noise, actuator dynamics, and calibration challenges; university competitions require investments exceeding $200,000 and limit participation to small teams; and small-scale robotic platforms, while cost-effective, fail to instill the safety-critical mindset that emerges from operating a vehicle capable of causing harm.

To address these limitations, we developed the platform at California State Polytechnic University, Pomona by converting a 350-pound electric utility vehicle into a fully instrumented drive-by-wire research testbed for approximately $12,000–15,000—roughly one-tenth the cost of commercial autonomous vehicle platforms. Unlike commercial solutions or competition vehicles, every layer of the platform—from low-level actuator firmware to high-level planning algorithms—is accessible, documented, and designed for curriculum integration.

The platform's technical architecture centers on a distributed Controller Area Network (CAN) with four Teensy 4.1 microcontrollers implementing closed-loop control on all actuators: steering via stepper motor with shaft encoder feedback, throttle through electronic control with wheel speed feedback, and braking using a linear actuator with position sensing. Students designed the entire communication protocol, message formats, and safety features including watchdog timers and fail-safe behaviors. The sensor suite comprises industry-grade hardware—an Xsens MTi-680G providing RTK-capable GPS/INS with centimeter-level positioning, a Velodyne VLP-16 LiDAR for 360-degree obstacle detection, and Intel RealSense depth cameras—exposing students to the same calibration challenges encountered in professional development. The modular Python software stack implements a sense-plan-control-act pipeline supporting multiple algorithms, from probabilistic occupancy mapping to Dynamic Window Approach local planning to Pure Pursuit path following, enabling comparative studies impossible on single-purpose platforms.

The platform supports six structured laboratory activities progressing from safety system familiarization through sensor integration, perception algorithms, motion planning, and culminating in team-based autonomous navigation field tests. Each laboratory engages students with production code rather than simplified teaching examples, enabling examination, modification, and extension of real systems.

To date, fourteen students across three semesters have engaged with the platform through mechanical design projects, embedded systems laboratories, and autonomous navigation research. We are implementing structured pre/post surveys measuring self-reported confidence in systems integration, hardware debugging, and safety-critical development practices. Preliminary observations indicate that students develop competencies employers consistently identify as lacking: systems thinking emerges when changes in one subsystem propagate through others; hardware debugging skills develop through diagnosing real sensor noise and communication issues; and a safety-critical mindset forms when software controls a vehicle capable of causing harm.

Our platform demonstrates that meaningful autonomous vehicle education does not require commercial platforms or exclusive competition programs. We offer it as a replicable model for institutions seeking to bridge the gap between academic preparation and the integrated systems competencies that industry demands, with complete documentation and source code available for adoption.

***Index Terms*—autonomous vehicles, engineering education, drive-by-wire, CAN bus, closed-loop control, hands-on learning, cyber-physical systems**

I. Introduction

The autonomous vehicle industry's rapid growth has created urgent demand for engineers who can integrate sensing, computation, and actuation into safety-critical systems. Yet industry leaders report that graduates, while technically competent in isolated domains, lack systems-level integration experience. A National Academies study found that Ford needs more cyber-physical systems engineers "than we can get," while NASA's Jet Propulsion Laboratory reported that 80% of new hires require substantial internal development due to gaps in integrated systems competencies [2]. The World Economic Forum identifies this skills gap as a major barrier to business transformation [1].

Current approaches to autonomous vehicle education each carry significant limitations. Simulation platforms like CARLA and Gazebo democratize access but abstract away real-world complexity—sensor noise, actuator dynamics, and calibration challenges that define physical systems development [3]. University competitions such as the SAE AutoDrive Challenge provide authentic experiences but require investments exceeding $200,000, limit access to competition teams, and often use proprietary systems that obscure learning opportunities [5]. Small-scale platforms like F1TENTH and

Duckietown offer cost-effective alternatives but sacrifice the engineering challenges of full-scale systems: tire dynamics, braking distances, and the safety-critical mindset that comes from operating a vehicle capable of causing harm [6], [7]. Commercial research platforms provide full-scale authenticity but at costs exceeding $150,000, accessible only to well-funded research groups.

This paper presents an autonomous vehicle platform developed at California State Polytechnic University, Pomona by converting a 400-pound utility terrain vehicle into a complete drive-by-wire research testbed. Unlike commercial solutions or competition vehicles, every layer of the platform—from low-level actuator firmware to high-level planning algorithms—is accessible, documented, and designed for curriculum integration.

The platform makes three primary contributions. First, we demonstrate a complete drive-by-wire conversion with custom-designed mechanical mounts and closed-loop control on all actuators (steering, throttle, brake), providing students hands-on experience with electromechanical systems integration. Second, we present a distributed embedded control architecture built on CAN bus, where students designed the communication protocol, implemented real-time firmware, and developed multi-layer safety systems. Third, we describe a modular sensor and software architecture supporting multiple perception and planning algorithms, enabling comparative studies impossible on single-purpose platforms. We offer the platform as a replicable model for institutions seeking meaningful autonomous vehicle education at a fraction of commercial costs.

II. Platform Design

A. Base Vehicle Selection

The base platform is an electric youth utility terrain vehicle powered by a 48V lithium battery system driving a 1.8 kW motor. The vehicle has a net weight of approximately 350 pounds, a 1.23-meter wheelbase, double A-arm front suspension, and rear hydraulic disc brakes. This platform was selected for several reasons: acquisition cost of approximately $3,000, electric drivetrain eliminating engine complexity while providing a 48V power bus for instrumentation, inherent safety features including roll cage and electronic speed limiting to 27 mph, physical accessibility for undergraduate students, and sufficient payload capacity (440 lbs) for sensors and computing hardware. Fig. 1 shows the completed platform with sensors mounted.

B. Drive-by-Wire Conversion

Converting the vehicle to drive-by-wire required designing and fabricating custom electromechanical systems for steering, throttle, and braking—each with encoder feedback enabling true closed-loop control. A key design principle was exposing complexity rather than hiding it: students can trace the complete signal path from a steering command issued by planning software through CAN bus transmission, microcontroller processing, motor driver actuation, and encoder feedback. This transparency enables debugging at any level of the system and supports learning objectives across mechanical, electrical, and software engineering curricula. The conversion process itself became a significant educational experience, requiring students to apply mechanical design, power transmission, and control systems concepts to a real vehicle.

Fig. 1. The autonomous vehicle platform: a converted electric UTV with LiDAR, RTK-GPS, depth camera, and onboard compute.

Steering. A stepper motor drives the steering column through a custom-designed mounting bracket fabricated to attach directly to the steering shaft. An encoder mounted on the shaft provides absolute position feedback, enabling closed-loop position control with sub-degree accuracy. The stepper motor was chosen for its holding torque and precise positioning without requiring a gearbox.

Throttle. The original mechanical throttle linkage was replaced with electronic throttle control. A wheel speed encoder provides velocity feedback, enabling closed-loop speed control. The control system accepts velocity setpoints and regulates throttle position to maintain the commanded speed, abstracting low-level throttle management from higher-level autonomy software.

Brake. A linear actuator was integrated with the existing hydraulic brake system, pressing the brake pedal mechanism under electronic control. An encoder on the actuator provides position feedback for closed-loop control. The brake system implements fail-safe behavior: loss of communication signal triggers immediate brake application, ensuring the vehicle stops safely if the control system fails.

Table I summarizes the closed-loop configuration for each actuator subsystem.

TABLE I
Actuator Closed-Loop Configuration

System	Actuator	Feedback	Control
Steering	Stepper motor	Shaft encoder	Position
Throttle	Electronic	Wheel encoder	Velocity
Brake	Linear actuator	Position encoder	Position

C. Embedded Control Architecture

The vehicle's control system is built on a distributed Controller Area Network (CAN) architecture with four Teensy 4.1 microcontroller nodes communicating at 250 kbps. The master node handles command parsing from higher-level software and safety arbitration. Three actuator nodes control steering (CAN ID 0x200), throttle (0x100), and braking (0x300), each running closed-loop control algorithms with encoder feedback. Students designed the entire communication protocol from scratch, defining message formats, implementing real-time firmware, and developing safety features including watchdog timers and fail-safe behaviors.

This architecture provides significant educational value. Unlike commercial platforms where students interact with pre-built APIs, our platform exposes the complete embedded systems stack. Students can observe CAN traffic, modify control loop gains, implement new message types, and debug timing issues—experiences directly applicable to automotive and robotics industry positions. Fig. 2 illustrates the distributed CAN architecture.

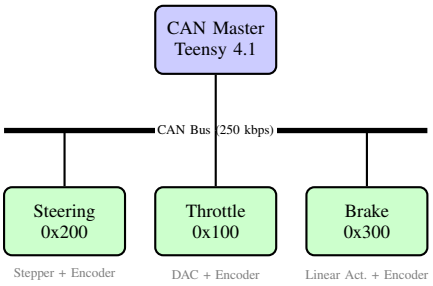

Fig. 2. Distributed CAN bus architecture with four Teensy 4.1 nodes. Each actuator node implements closed-loop control with encoder feedback.

D. Network Infrastructure

A network router serves as the communication backbone, connecting the Jetson Orin AGX (primary compute for GPU-accelerated perception), the CAN master node, and any development laptops via Ethernet (Fig. 3). This architecture deliberately avoids tying the platform to a single computer. Students can connect their personal laptops to develop and test code, view sensor data, or manually control the vehicle through a web-based interface. The same backend supports multiple control modalities: joystick control via web UI, voice commands for hands-free operation, and the full autonomous navigation stack.

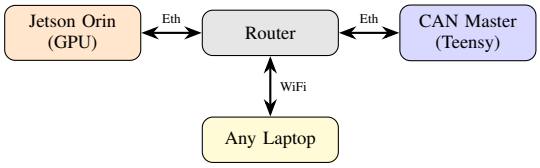

Fig. 3. Network topology enabling flexible development. Any laptop can connect to develop, test, or control the vehicle.

E. Modular Electronics

Power distribution flows from the vehicle's 48V lithium battery through three DC-DC converters, each feeding its own dedicated fuseblock. Two 48V-to-12V converters provide isolated power domains: one powers the actuator systems (steering motor, throttle controller, brake actuator), while the other powers perception hardware (LiDAR, Jetson compute). This separation ensures that electrical faults in motor drive circuits cannot propagate to sensitive sensor electronics—a deliberate safety decision that students learn to appreciate when debugging intermittent sensor issues.

A dedicated 48V-to-5V converter with its own fuseblock powers the Teensy microcontrollers and encoder electronics, providing a clean logic-level supply isolated from the higher-current 12V domains. Each fuseblock allows individual circuits to be safely isolated during development, enabling students to power only the subsystems they are testing. This modularity reduces risk during debugging sessions and teaches practical lessons about power system design in embedded applications.

The architecture is explicitly designed for expansion—additional CAN nodes, sensors, or actuators can be added without redesigning the power distribution system, supporting the platform's evolution as research needs grow.

III. Sensing & Software

The sensor suite comprises industry-grade hardware that exposes students to the same calibration challenges, data formats, and accuracy considerations encountered in professional autonomous vehicle development. Table II summarizes the sensor specifications.

A. Sensor Suite

TABLE II
Sensor Suite Specifications

Sensor	Model	Key Specs	Function
GPS/IMU	Xsens MTi-680G	RTK: ±5–10 cm*; 9-axis; 100 Hz	Localization
LiDAR	Velodyne VLP-16	16 layers; 360°; 100 m; 10 Hz	Obstacles
Depth Cam	Intel RealSense	640×480 RGB-D; 30 fps	Close-range
Compute	Jetson Orin	GPU; Ethernet	Perception

*RTK accuracy with correction; degrades to ±2–5 m otherwise

The Xsens MTi-680G provides RTK-capable GNSS/INS positioning with centimeter-level accuracy when RTK correction is available, degrading gracefully to meter-level accuracy otherwise. The sensor outputs position in WGS84 coordinates, velocity and orientation in ENU (East-North-Up) frame at 100 Hz, and reports RTK fix status enabling students to monitor positioning quality in real-time. The General_RTK filter profile was selected to avoid magnetic interference from the metal vehicle chassis, relying purely on GNSS and inertial fusion for heading estimation. Students examine

the `XsensReceiver` class to understand coordinate frame transformations and thread-safe sensor data management.

The Velodyne VLP-16 provides 360-degree coverage with 16 scanning layers spanning ± 15 degrees vertically. At 10 Hz scan rate with 100-meter range and ± 2–3 cm accuracy, the sensor generates approximately 300,000 points per second. The UDP-based interface exposes students to network protocols and packet parsing—skills directly applicable to other industrial sensors. Point clouds feed the occupancy grid mapping pipeline for obstacle detection.

Intel RealSense depth cameras provide aligned RGB-D data with factory-calibrated intrinsics, enabling students to focus on perception algorithms rather than calibration procedures. Depth measurements at millimeter resolution support close-range obstacle detection and camera-based scene understanding. The software architecture also supports multi-task neural network perception for lane detection and drivable area segmentation, though the primary perception pipeline relies on LiDAR for obstacle avoidance.

B. Perception Pipeline

The perception layer provides reference implementations that students can use directly, modify, or replace entirely with their own algorithms. The default occupancy grid uses the log-odds Bayesian framework: each cell maintains a belief that is incrementally updated as observations arrive, with `log_odds_occ` (default 0.4) for occupied cells and Bresenham raycasting marking free space with `log_odds_free` (default -0.2). A temporal decay factor causes beliefs to fade toward uncertainty, preventing stale data. The default 40m $\times$ 40m grid at 0.1m resolution suits low-speed navigation, but students can adjust grid dimensions, resolution, and all update parameters to observe effects on detection latency, false positive rates, and convergence behavior.

For motion planning, the probabilistic occupancy grid is transformed into a binary costmap through obstacle inflation. Cells exceeding an occupancy threshold are marked as obstacles, then dilated by the robot radius plus a configurable safety margin using morphological operations. This inflation converts the world representation into configuration space, where a point robot can safely plan without explicit collision geometry. The `Costmap` class provides efficient single-point collision queries used by the DWA planner.

The perception architecture supports camera-based scene understanding through inverse perspective mapping (IPM) and multi-task neural networks. IPM transforms camera images to bird's-eye view using precomputed homography for real-time performance. While the primary obstacle avoidance relies on LiDAR, the vision pipeline provides lane detection capability for future curriculum modules.

C. Software Architecture

The software architecture follows a modular sense-plan-control-act pipeline (Fig. 4). In the default configuration, GPS/IMU streams at 100 Hz, LiDAR at 10 Hz, and the control loop at 20 Hz—but all rates are configurable, and students frequently experiment with different timing strategies. This separation allows students to modify any layer independently—implementing a new planner requires only conforming to the waypoint interface, not understanding sensor drivers or actuator protocols. The codebase uses Python with type hints and documentation, prioritizing readability over optimization.

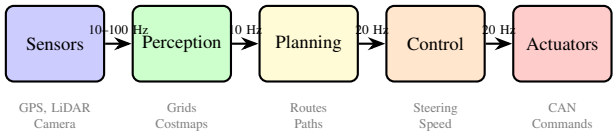

Fig. 4. Modular software pipeline showing default operating frequencies. All rates are configurable; students can modify any layer independently while interfaces remain stable.

Real-time operation requires careful thread management. The control thread runs in background at 20 Hz, reading sensor state, computing steering commands, and transmitting actuator commands. A `VehicleState` class provides thread-safe data sharing via lock-protected getters and setters, enabling atomic snapshots that prevent partial reads during control computations. The visualization thread runs in the main process at lower priority, updating displays without blocking control. This architecture teaches practical patterns—producer-consumer synchronization, lock granularity, and non-blocking design—that students encounter in professional robotics software.

The planning layer supports multiple algorithms: global route planning using road network graphs, local reactive planning using the Dynamic Window Approach, and geometric path following using Pure Pursuit with adaptive lookahead. This algorithm diversity enables comparative studies—students can evaluate different approaches on the same physical platform under identical conditions.

D. Accessible API Design

To lower the barrier to entry while preserving learning depth, the platform provides a Python API that abstracts hardware complexity without hiding it. Sensor classes inherit from a common `SensorInterface` base class providing standardized lifecycle methods (connect, start, stop, disconnect), thread-safe data access, and callback registration. The actuator interface is equally streamlined: controlling the vehicle requires only instantiating `VehicleActuatorUDP` and calling methods like `set_throttle(0.5)` or `set_steer_norm(-0.2)`. Students can begin by treating these as black boxes, then progressively examine the implementation—UDP packet formatting, CAN message construction, encoder feedback—as their understanding grows. This layered design supports the scaffolded learning progression described in Section IV.

IV. Demonstration Results

The platform has been validated through autonomous navigation tests on campus. Table III summarizes key performance metrics from field testing.

```python
from drive import Vehicle, GPS

# Initialize hardware interfaces
vehicle = Vehicle()
gps = GPS()

# Basic control
vehicle.set_throttle(0.5)    # 50% forward
vehicle.set_steering(-0.3)   # Turn left

# Read sensor data
position = gps.get_position()
heading = gps.get_heading()
```

Listing 1. Simplified API example for vehicle control and sensor access.

TABLE III
PLATFORM PERFORMANCE METRICS

Metric	Value
Cross-track error (mean)	*TBD*
Localization accuracy (RTK)	±5–10 cm
Maximum autonomous speed	*TBD*
Control loop frequency	20 Hz
Watchdog timeout	500 ms
Continuous operation time	*TBD*

The platform successfully demonstrates GPS waypoint following using Pure Pursuit control, obstacle detection via LiDAR-based occupancy mapping, and fail-safe behavior through watchdog-triggered braking. Detailed performance characterization from systematic field trials will be reported in the full paper.

V. CURRICULUM INTEGRATION

The platform was designed explicitly for curriculum integration across multiple engineering disciplines, with its pedagogical approach grounded in established learning theory. To date, fourteen students across three semesters have engaged with the platform, with usage spanning mechanical design projects, embedded systems laboratories, and autonomous systems research.

A. Pedagogical Approach

The curriculum follows experiential learning principles [9], with laboratory activities progressing from hands-on operation through structured analysis to independent implementation—an approach shown effective in robotics education [6], [10].

B. Multidisciplinary Integration

The platform's modular architecture enables targeted engagement across engineering disciplines at multiple levels. Introductory courses use the platform for demonstrations of cyber-physical systems concepts. Embedded Systems courses focus on CAN bus communication, actuator interfaces, and real-time firmware development. Robotics and AI courses emphasize perception algorithms and motion planning. Control Systems courses leverage the closed-loop actuators for PID tuning on physical hardware. Senior Design projects use the complete platform for capstone experiences integrating mechanical, electrical, and software engineering.

C. Laboratory Activities

The platform supports six structured laboratories progressing from foundational safety concepts to full system integration (Table IV). Each laboratory engages students with production code rather than simplified teaching examples, and culminates in a team-based field test of autonomous navigation.

TABLE IV
LABORATORY ACTIVITIES

Lab	Title	Key Tasks
1	Platform Safety	Trace E-STOP signal; test watchdog timeout
2	GPS/IMU	Configure Xsens; transform coordinates
3	Occupancy Mapping	Tune log-odds; implement raycasting
4	Planner Comparison	Compare DWA and Pure Pursuit metrics
5	Controller Tuning	Tune Pure Pursuit lookahead; PID gains
6	System Integration	Field test autonomous navigation (team)

VI. ASSESSMENT & OUTCOMES

A. Skills Developed

Working with the platform develops competencies that employers consistently identify as lacking in new graduates. *Systems thinking* emerges naturally when students observe how a change in one subsystem propagates through others—a modified control gain affects not just actuator response but also planning behavior and safety margins. *Hardware debugging skills* develop through diagnosing real sensor noise, communication dropouts, and mechanical issues that never appear in simulation. *Applied control theory* becomes tangible when students tune PID gains on physical actuators and observe overshoot, oscillation, and steady-state error on a real system. Perhaps most importantly, students develop a *safety-critical mindset*: writing software that controls a 350-pound vehicle changes how one thinks about error handling, testing, and validation.

B. What Hardware Teaches

Physical systems exhibit behaviors that simulation abstracts away. Encoder readings contain noise and drift over time. Actuators have deadbands where small commands produce no motion, and saturation limits where larger commands produce no additional effect. Mechanical systems exhibit backlash and play. Communication buses occasionally drop messages. These phenomena are not edge cases—they are the defining challenges of real-world autonomous systems. Students who encounter them on the vehicle develop intuition and debugging strategies that transfer directly to industry positions. The physical consequences of software bugs—a vehicle that steers unexpectedly or fails to brake—create learning moments that simulation cannot provide.

C. Assessment Instrument

To quantify learning outcomes, we are deploying a pre/post survey to students who engage with the platform. The instrument measures self-efficacy across key competency areas using a 5-point Likert scale:

- "I can debug issues that span hardware and software boundaries"
- "I can trace a signal path from sensor through processing to actuator"
- "I understand how sensor noise affects system behavior"
- "I feel prepared to work on real robotic or autonomous systems"

Open-response questions capture qualitative insights: "What did working with physical hardware teach you that simulation could not?" and "Describe a debugging experience that changed how you approach problems." Results from current and former students (n=14+) will be reported in the full paper.

VII. Discussion

A. Effective Design Choices

Several architectural choices proved particularly effective. Router-based networking enabled flexible development—any laptop can connect to the system, eliminating dependence on a single development machine. The fuseblock power distribution system allows safe isolation of subsystems during testing and development. Placing encoders on all actuators enabled true closed-loop control, transforming what could have been open-loop demonstrations into proper feedback control systems. The Teensy 4.1 microcontroller offered an excellent balance of capability, cost, and accessibility, with extensive documentation and an active community.

B. Challenges

Custom mechanical mounts required access to fabrication equipment (3D printing, machining), which may limit replicability at institutions without maker spaces or machine shops. Initial CAN bus debugging presented a steep learning curve, though this challenge itself became educational. Outdoor testing depends on weather conditions, occasionally limiting platform availability.

C. Cost Analysis

Table V presents approximate costs for replicating the platform. The total investment of approximately \$12,000–15,000 is roughly one-tenth the cost of commercial autonomous vehicle research platforms, while providing complete access to all system layers.

D. Comparison with Other Platforms

Table VI compares our platform with other educational autonomous vehicle platforms. Our platform occupies a unique position: it provides full-scale vehicle dynamics and safety-critical experience at a fraction of commercial costs, while offering complete system transparency unavailable in competition platforms.

TABLE V
Approximate Platform Costs

Component	**Cost (USD)**
Base vehicle	\$3,000
Sensors (LiDAR, GPS, cameras)	\$4,000–6,000
Compute (Jetson Orin AGX)	\$2,000
Electronics (Teensy, actuators, wiring)	\$1,500
Fabrication and miscellaneous	\$1,500
Total	\$12,000–15,000

TABLE VI
Comparison with Educational AV Platforms

Platform	**Scale**	**Cost**	**Sensors**	**Open**	**DBW**
Ours	Full	\$12–15k	RTK-GPS, LiDAR	Yes	Yes
F1TENTH	1:10	\$3–4k	2D LiDAR	Yes	No
Duckietown	Small	\$150–300	Camera	Yes	No
MuSHR	1:10	\$1–2k	2D LiDAR	Yes	No
AutoDrive	Full	\$200k+	Full suite	No	Yes

DBW = Drive-by-wire with closed-loop control. Open = Open-source/accessible.

Small-scale platforms like F1TENTH and Duckietown excel at algorithm development but cannot replicate the engineering challenges of full-scale systems: real tire dynamics, consequential braking distances, and the safety-critical mindset that emerges from controlling a vehicle capable of causing harm. Competition platforms like AutoDrive provide authentic full-scale experience but at costs exceeding \$200,000 and with proprietary systems that limit learning opportunities.

E. Replication Resources

To support adoption at other institutions, we are preparing comprehensive documentation including: a complete bill of materials with supplier links, CAD files for custom mechanical mounts, firmware source code for Teensy CAN nodes, the Python software stack with API documentation, and a step-by-step build guide. These resources will be released as open source at `github.com/Paarseus/AV2.1-API` upon publication.

VIII. Conclusion

This work demonstrates that meaningful autonomous vehicle education does not require commercial platforms costing hundreds of thousands of dollars or exclusive competition programs. By converting an electric utility vehicle into a complete drive-by-wire research platform with custom CAN architecture, closed-loop control on all actuators, and a modular sensor and software stack, we created an educational tool where every layer—from embedded firmware to planning algorithms—is accessible and modifiable.

The platform addresses a real gap in engineering education: producing graduates who can integrate across mechanical, electrical, and software domains to build safety-critical cyber-physical systems. Students working with the vehicle develop

systems thinking, hardware debugging skills, and a safety-critical mindset that simulation-only approaches cannot provide.

Future work includes ROS 2 integration to align with industry-standard middleware, open-source release of firmware and software to support replication at other institutions, expansion to a multi-vehicle fleet for coordination research, and development of formal assessment instruments to measure learning outcomes quantitatively. We offer this platform as a replicable model for institutions seeking to establish autonomous vehicle education programs that bridge the gap between academic preparation and industry needs.

Acknowledgment

This work was supported by the Ganpat and Manju Center for International Collaboration and Innovation at Cal Poly Pomona, established through the generous contribution of Dr. Ganpat Patel and Manju Patel. The platform was developed as part of the iCARE-M&S (Industry 4.0: Career Advancement through Research and Education in Modeling and Simulation) program, which aims to increase the pool of STEM professionals with expertise in autonomous systems. The authors thank the members of the Autonomous Vehicle Laboratory for their contributions to platform development.

References

[1] World Economic Forum, "The Future of Jobs Report 2025," Geneva, Switzerland, 2025.

[2] National Academies of Sciences, Engineering, and Medicine, "A 21st Century Cyber-Physical Systems Education," Washington, DC: The National Academies Press, 2016.

[3] M. Rahman, S. Ahmed, and K. Patel, "Effectiveness of Virtual Laboratory in Engineering Education: A Meta-Analysis," *PLOS ONE*, vol. 19, no. 3, 2024.

[4] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2017, pp. 23–30.

[5] J. M. Bastiaan, D. L. Peters, J. R. Pimentel, and M. Zadeh, "The AutoDrive Challenge: Autonomous Vehicles Education and Training Issues," in *Proc. ASEE Annual Conf. Expo.*, Tampa, FL, 2019.

[6] J. Betz *et al.*, "F1TENTH: Enhancing Autonomous Systems Education Through Hands-On Learning and Competition," *IEEE Trans. Educ.*, 2024.

[7] L. Paull *et al.*, "Duckietown: An Open, Inexpensive and Flexible Platform for Autonomy Education and Research," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 1497–1504.

[8] S. S. Srinivasa *et al.*, "MuSHR: A Low-Cost, Open-Source Robotic Racecar for Education and Research," *arXiv preprint arXiv:1908.08031*, 2019.

[9] D. A. Kolb, *Experiential Learning: Experience as the Source of Learning and Development*. Englewood Cliffs, NJ: Prentice-Hall, 1984.

[10] M. Abdulwahed and Z. K. Nagy, "Applying Kolb's experiential learning cycle for laboratory education," *J. Eng. Educ.*, vol. 98, no. 3, pp. 283–294, 2009.